Selecting Reliable LLM Reasoning: Arithmetic Evidence

DASE7506 MP2 | Follow-up experiment | 31 October 2026

This experiment asks whether inexpensive, mechanically checkable arithmetic evidence improves candidate selection beyond the supplied representations and trace statistics. It adds eighteen deterministic features describing numerical equation consistency, then chooses a compact linear ranker using training-only question-level cross-validation. No new language-model generation, external dataset or pretrained weights are used.

Development result	Original submitted verifier	Arithmetic candidate
Training OOF, seed 2026	90.81% (4318/4755)	91.65% (4358/4755)
Reshuffled train OOF, seed 2037	90.75%	91.84%
Validation	92.23% (736/798)	94.24% (752/798)
Public test score	81.37% (380/467)	Not evaluated this round

The result supports an improvement on development evaluations, not a new leaderboard claim. The original official submission #340 is left unchanged. The arithmetic candidate is separately frozen and reproducible; no public test inputs or references are opened during this round.

Task, data and protocol

Each question has four supplied candidate solutions. Select one candidate index; the unchanged official scorer checks its final numeric answer, not the validity of every reasoning step. The bank has 4,755 training, 798 validation and 467 test questions. All groups contain a correct final answer, and several candidates can be correct. Only training references fit model parameters and normalization. Validation is inspected after the current configuration is selected, without a validation parameter search.

The original baseline is the training-standardized C=0.01 logistic verifier over embeddings, centered embeddings and 46 trace/answer statistics. Previous CPU and GPU rounds are retained as historical evidence. Their observed results and repeated use of the course development bank mean this report should not characterize the present estimates as a fresh independent study.

Main evidence

The selected ranker gains forty training-OOF questions over the original verifier, with the direction retained under a reshuffled partition. A matched loss/regularization ablation gains twenty questions when arithmetic features are added. The parser checks at least one numeric equality in 63.57% of training candidate traces, but it deliberately skips many unsupported expressions. Arithmetic consistency is informative without being a proof of correct interpretation.

1. Bounded arithmetic checks and ranker

Conservative numeric extraction

The checker normalizes supported LaTex fractions, multiplication/division signs, parentheses, percentages and grouped thousands. It examines adjacent sides of equality signs only when numeric expressions can be extracted. A bounded AST interpreter accepts numeric constants and arithmetic operators; it does not execute Python eval, calls, names, indexing or attributes. Expression length, recursion depth, exponent size and fraction bit length are capped. Division by zero and unsupported syntax are rejected.

Checks cover fractions, decimals, signs, powers and percentages. A comparison is accepted within 1% relative tolerance plus 1e-6; a literal decimal right side additionally allows half a unit of its displayed precision. This accommodates common rounded answers. Unit annotations and unknown algebraic expressions can cause the equality to be skipped. The feature extractor is a heuristic span parser, so lack of a flag is not evidence that a trace is correct.

Feature group	Examples
Counts and coverage	Checked, inconsistent and consistent equations; raw equalities; supported coverage
Severity and final-answer relation	Inconsistency fraction; maximum/mean relative discrepancy; last check; checks sharing the final numeric answer
Structure and agreement	Division and exact-equality fractions; relative inconsistency versus peers; mean inconsistency for candidates sharing an answer

Eighteen arithmetic features are appended to the original 1,024-dimensional embedding and 46 statistics, giving 1,088 input dimensions. No arithmetic result is used as a hard veto: a trace with a bad step can still have a correct final answer, and parsing can be incomplete. Training determines how much predictive weight these features receive.

Objective and inference

The selected model uses marginal listwise loss. For each informative training group, it minimizes logsumexp over valid candidate scores minus logsumexp over correct candidate scores, plus 0.1 times squared weight norm divided by two. Groups with no valid wrong candidate contribute no selection information after masking and are excluded from this fitting loss. StandardScaler uses training candidates only; L-BFGS-B fits a single linear weight vector with zero intercept. Prediction chooses the largest score among candidates that the official parser can read.

```
x_ij = [embedding_ij, original_statistics_ij, arithmetic_ij]
z_ij = w^T ((x_ij - training_mean) / training_scale)
loss_i = logsumexp(valid z_i) - logsumexp(correct z_i)
choice_i = argmax_j z_ij     (invalid parsed answers masked)
```

The numeric checker does not understand the question, units or physical meaning. For example, 94 - 46 = 48 is arithmetically true even if 46 counts pairs while 94 counts people. It can only provide partial evidence for the full selection task.

2. Comparisons, ablation and uncertainty

The recorded search compares the original baseline and nine arithmetic candidates: three pointwise BCE regularization settings, three pairwise logistic settings and three marginal-listwise settings. All model selection uses three question-disjoint training folds with seed 2026. The original baseline wins ties. A preliminary parser-debug run is aborted before validation; partial-expression rejection tests are added and the corrected search is run in full. The partial debug log is retained separately.

Training-OOF configuration	Accuracy %
original	90.81
bce_context_0.001	90.96
bce_context_0.01	91.06
bce_context_0.1	90.54
pair_combined_0.001	91.38
pair_combined_0.01	91.52
pair_combined_0.1	91.13
marginal_combined_0.01	91.23
marginal_combined_0.1	91.65
marginal_combined_1.0	90.96

The arithmetic winner is marginal_combined_0.1. It fixes 166 original OOF errors and introduces 126, for a net gain of forty. A paired question bootstrap with 5,000 resamples and seed 2026 gives a descriptive gain interval of +0.15 to +1.54 percentage points. The interval does not correct for selecting among configurations, overlapping fold-training data or adaptation across earlier project rounds.

Diagnostic (same training bank)	Control	Selected candidate
Matched objective/regularization; omit arithmetic	91.23% (4338/4755)	91.65% (4358/4755)
Reshuffled three-fold diagnostic, seed 2037	90.75% original verifier	91.84% arithmetic ranker

The matched feature ablation holds the loss, regularization, numeric parser and folds constant; adding arithmetic features gains twenty questions (+0.42 points). The complete method versus the original verifier also changes its training objective, so the full +0.84-point gain cannot be attributed only to arithmetic features. The reshuffled diagnostic evaluates the already selected method and does not trigger another configuration search. It reuses the same questions, so it is not an external audit.

Only after training-OOF selection and full training is the selected configuration evaluated on validation: 752/798 = 94.23558897%. This matches the highest validation score seen in an earlier CPU round, but the methods and selection protocols differ. The earlier method scored worse on its final test; therefore equal validation accuracy is not evidence of equal or better test accuracy. No new test score is claimed.

3. Limits, resources and reproduction

What arithmetic evidence misses

In training, at least one inconsistency is flagged in 1.63% of candidates with a correct final answer and 6.02% of candidates with a wrong final answer. The association is useful but imperfect: erroneous intermediate steps can cancel, a final answer can be correct by chance, and extraction or rounding heuristics can be incomplete. These rates describe candidate-level final-answer labels, not human-verified step correctness.

Post-selection validation diagnostics illustrate the limits. For the stock-profit question at position 48, the original verifier selects 100 and the new pipeline selects the reference answer 40; this change cannot be causally attributed to the arithmetic features alone. At position 52, the dog/rabbit question should yield four minutes, but both methods still select eight. The relevant unit-bearing equations are skipped by the conservative parser. At position 44, both still select 48 for the pairing problem, because its arithmetic is valid despite counting the wrong quantity. The ambiguous paperclip reference at position 32 also remains unresolved.

Resources and integrity checks

All fitting in this round uses CPU, with four numerical-library threads. The corrected search takes 131.98 s locally; the reshuffled diagnostic takes 56.73 s and matched ablation 19.26 s. Partial debug timing is retained in its separate log. The final required assets total 14,323 bytes. Fresh official CPU validation prediction plus scoring takes 1.896 s on an Intel Core i7-13650HX, including startup/imports, hashing, loading and preprocessing. No reference-Xeon measurement or new complete-test timing is claimed.

Behavioral numeric/LaTex/rounding/unit/unsafe-expression checks pass. Float32 export preserves every validation choice. Renamed IDs, a seven-question bank and candidate score permutations pass. The official data, numeric parser and scorer remain unchanged. Inference receives new bank/array inputs and never opens references, stored predictions or answer lookup tables. Frozen source/asset hashes are recorded in runs/freeze-arithmetic.json.

Exact commands from code/

```
python -m pip install -r requirements.txt
python download_data.py
python check_arithmetic.py
python evaluate_arithmetic.py

# Optional complete training and diagnostic reproduction
python -m pip install -r requirements-training.txt
python train_arithmetic.py
python check_robustness_arithmetic.py
python ablate_arithmetic.py
```

The inference assets are runs/model_bundle_arithmetic. Training-only selection and full records are in train_arithmetic.py and runs/arithmetic-search/. Dependencies remain NumPy 1.26.4, SciPy 1.13.1, scikit-learn 1.5.1, joblib 1.4.2 and threadpoolctl 3.5.0. Seeds are recorded in the plan and diagnostics. Checkpoint evaluation requires NumPy only.

AI/reuse disclosure: Codex substantially assisted with design, implementation, debugging, experiments, analysis and report writing. Official course tools are reused unchanged. Questions/references derive from GSM8K (retained MIT license); supplied candidates/embeddings derive from Qwen3-0.6B with revision in the manifest. No additional data or pretrained weights are used. Course source: github.com/xudongwu-0/DASE7506-MP2, upstream bb801f4f2e7629d270df765ab12c333b2dc616bb. The current candidate is an experiment and does not replace the existing official submission.